APS106

UNIVERSITY OF TORONTO

Introduction to Object Oriented Programming, and the file object.

Week 5 | Lecture 2 (5.2)

if nothing else, write #cleancode

This Week's Content

- **Lecture 5.1**
  - Strings: Conversions, Indexing, Slicing, and Immutability
- **Lecture 5.2**
  - **Strings: practice exercise**
  - **Introduction to Object-Oriented Programming and the File Object**
- **Lecture 5.3**
  - **for loops**

Chaining Methods Together

- How would you replace the word “forward” with “backward”?

```
>>> s = 'Forward, forward we must go, for there is no other way to go!'

>>> s_1 = s.lower()
'forward, forward we must go, for there is no other way to go!'
>>> s_2 = s_1.replace('forward', 'backward')
'backward, backward we must go, for there is no other way to go!'
>>> s_3 = s_2.capitalize()
'Backward, backward we must go, for there is no other way to go!'
```

Chaining Methods Together

- How would you replace the word “forward” with “backward”?

```
>>> s = 'Forward, forward we must go, for there is no other way to go!'
```

- Methods can be chained together
  - Perform first operation, which returns an object
  - Use the returned object for the next method

```
>>> s.lower().replace('forward','backward')
```

```
>>> s.lower().replace('forward','backward').capitalize()
```

More String Methods

- There are many more string methods available which you will explore in lab assignments and tutorials
  - `str.islower()`
  - `str.count(sub)`
  - `str.ljust(width)`
  - `str.lstrip()`
  - `str.split()`
  - `str.strip()`

Do not need to memorize all of the string methods!

Should know:

`str.lower`
`str.upper`
`str.find`
`str.rfind`
`str.replace`
`str.capitalize`
`str.startswith`

(any others will be indicated in the review)

Let's warm up!

- Let's take a look at how this works in Python!
  - Counting characters regardless of case

Breakout Session!

What is Object-Oriented Programming (OOP)?

To investigate, let's compare

Procedural Programming to **OOP**

Procedural Programming

Global Variable
Pedestrian 1
x, y Location

Global Variable
Pedestrian 2
x, y Location

Global Variable
Pedestrian 3
x, y Location

```
x_ped1 = 3
y_ped1 = 5
```

Global Variable
Traffic Light 1
Color

Global Variable
Car 1
x, y location

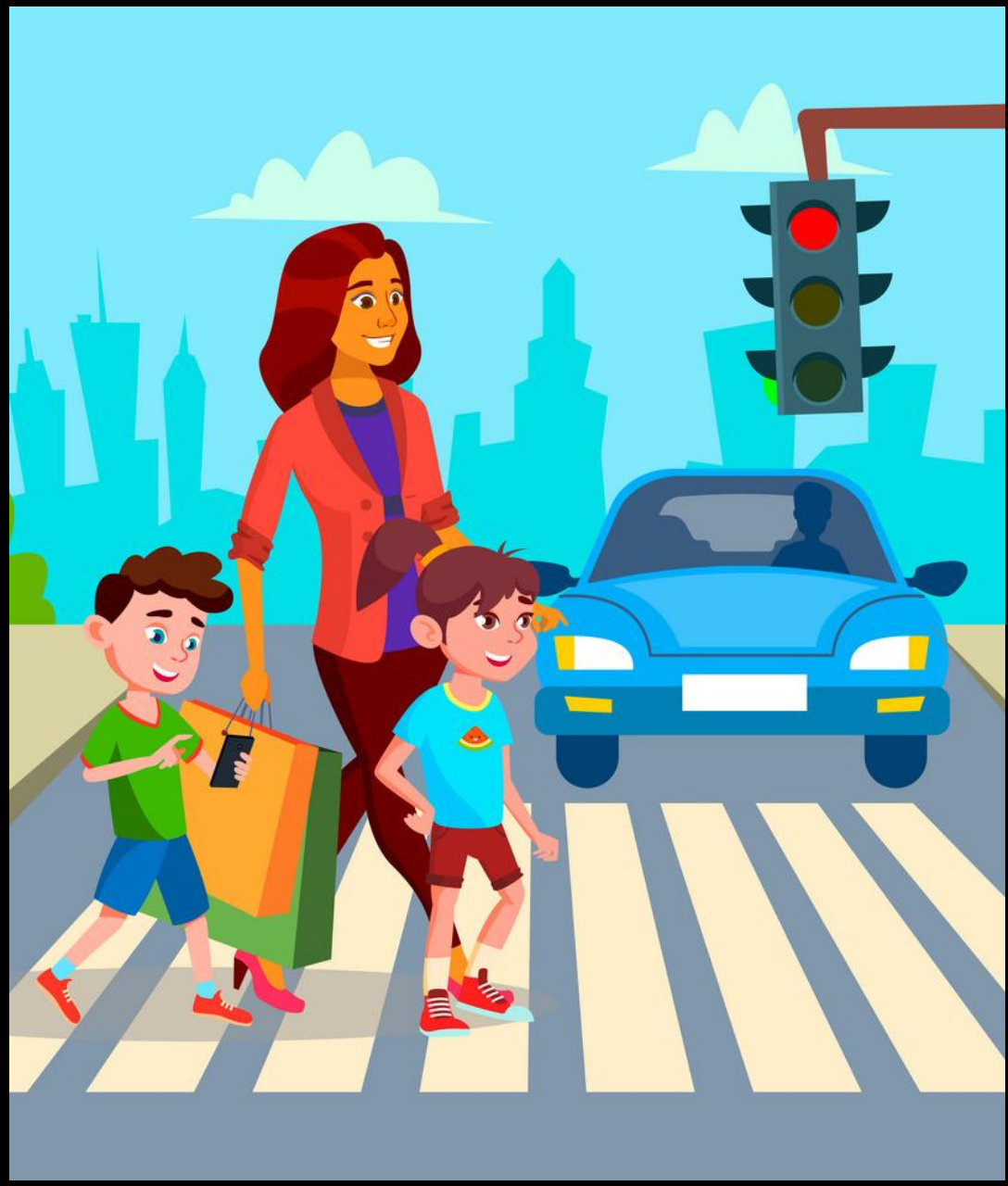
```
traffic_light = 'red'
```

Separation of Data and Functions

Function
`okToCross`

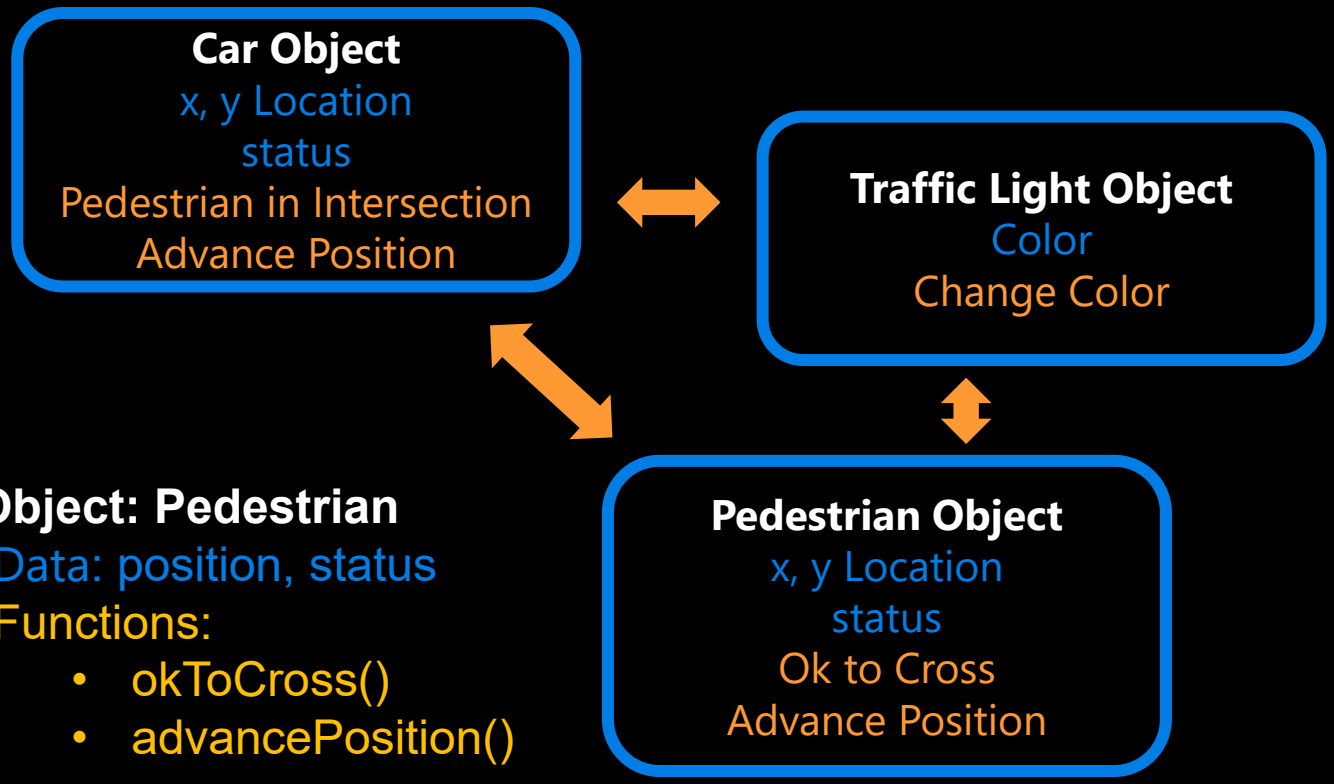
Function
`AdvancePosition`

Function
`isInIntersection`



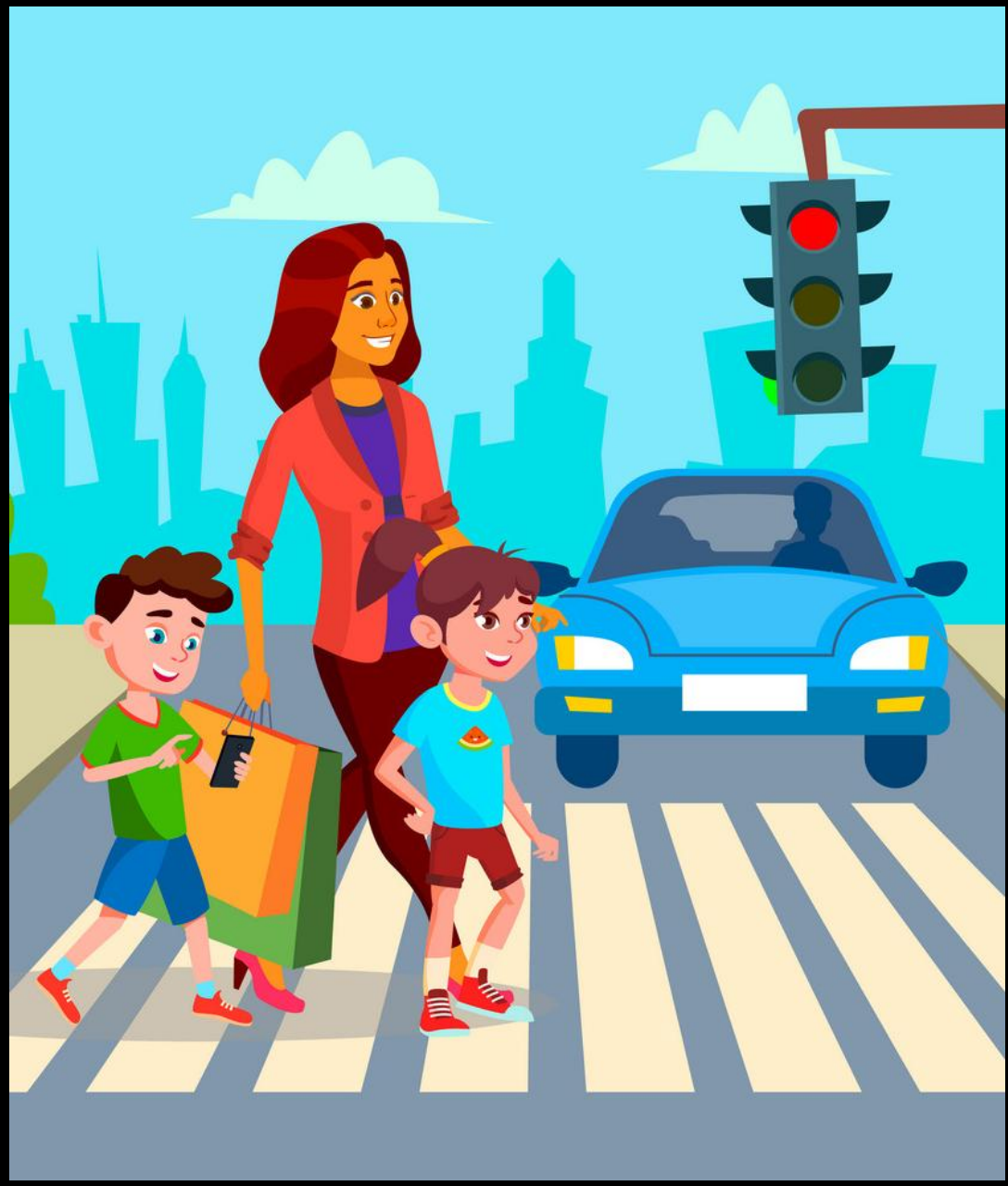
Object-Oriented Programming

Object-oriented programming bundle the Data together with these Functions into a single object



Object: Pedestrian

- Data: position, status
- Functions:
  - okToCross()
  - advancePosition()



Object-Oriented Programming

- Often, an object definition corresponds to some object or concept in the real world.
- The functions that operate on that object correspond to the ways real-world objects interact.

- Models our real-life thinking
  - Sandwich – ingredients, freshness, etc.
  - Car – model, year, fuel level, forward, reverse etc.
  - Cat – weight, name, colour, scratch, meow, sleep, etc.
  - Turtle Object – x_position, y_position, forward, up, left, etc.

Object-Oriented Programming

Data
Functions

Procedural

```
def up(y):
  return y + 1

def goto(x_new, y_new):
  return x_new, y_new

def right(x):
  return x + 1

alex_x = 0
alex_y = 0

alex_y = up(alex_y)
alex_x, alex_y = goto(-150, 100)
alex_x = right(alex_x)

print(alex_x, alex_y)
```

Object-Oriented

```
alex = Turtle(0, 0)

alex.up()
alex.goto(-150, 100)
alex.right()

print(alex.x, alex.y)
```

Objects in Python

- **Everything in Python is an object.**
- Every value, variable, function, etc., is an object.
- Every time we create a variable we are making a new object.

Is **this** an instance of **this** class.

```
>>> isinstance(4, object)
True

>>> isinstance(max, object)
True

>>> isinstance("Hello", object)
True
```

We've already been using objects!

integer

is_integer
as_integer_ratio
to_bytes
from_bytes
bit_count
...

string

replace
find
rfind
upper
isupper
islower
capitalize
count
...

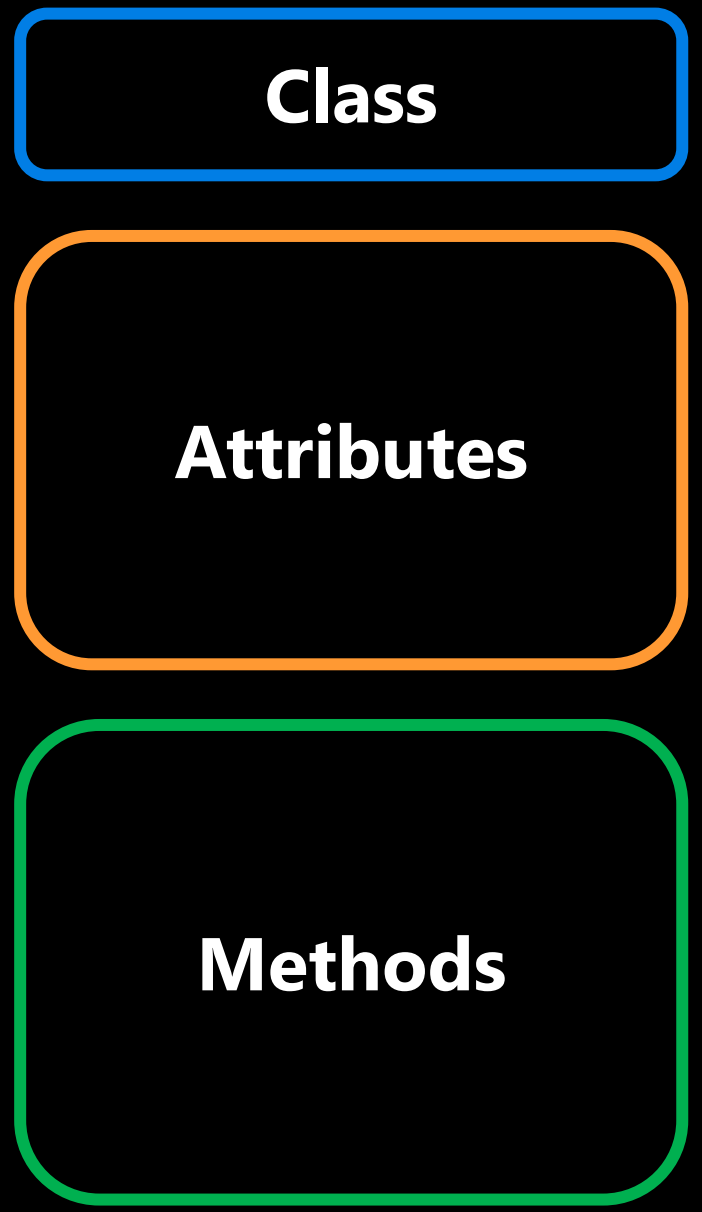
math

pi
e
inf
...

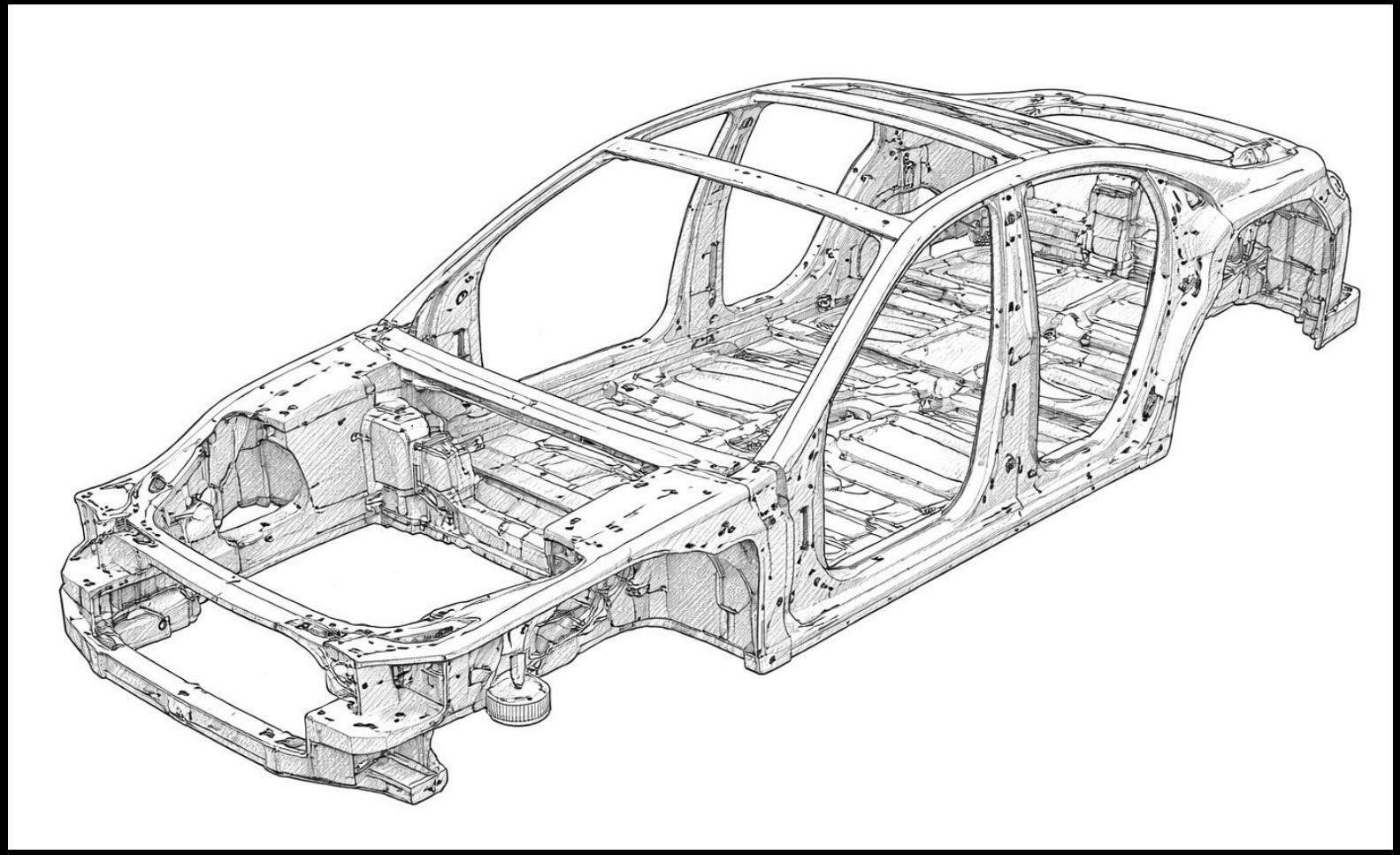
factorial
ceil
floor
isfinite
log
...

Classes

- A template or blueprint for object types
- An instance of a class refers to one object whose type is defined as the class.
- The words "instance" and "object" are used interchangeably.
- A Class is made up of attributes (**data**) and methods (**functions**).



Classes: blueprints



Car

model
year
tires
color

brake
accelerate
open trunk

model: Corolla
year: 2024
tires: None
color: red

Car

model
year
tires
color

brake
accelerate
open trunk

model: Corolla
year: 2024
tires: None
color: grey

Car

model
year
tires
color

brake
accelerate
open trunk

model: Corolla
year: 2024
tires: fancy
color: grey

Car

model
year
tires
color

brake
accelerate
open trunk

model: Corolla
year: 2024
tires: monster
color: grey

Car

model
year
tires
color

brake
accelerate
open trunk

model: Sierra
year: 2020
tires: monster
color: grey flame decals

Car

model
year
tires
color

brake
accelerate
open trunk

Let's Code!

- Let's take a look at how this works in Python!
  - Objects in Python
  - One potential ACORN design

In your notebook,

Click Link:
1. Objects in Python

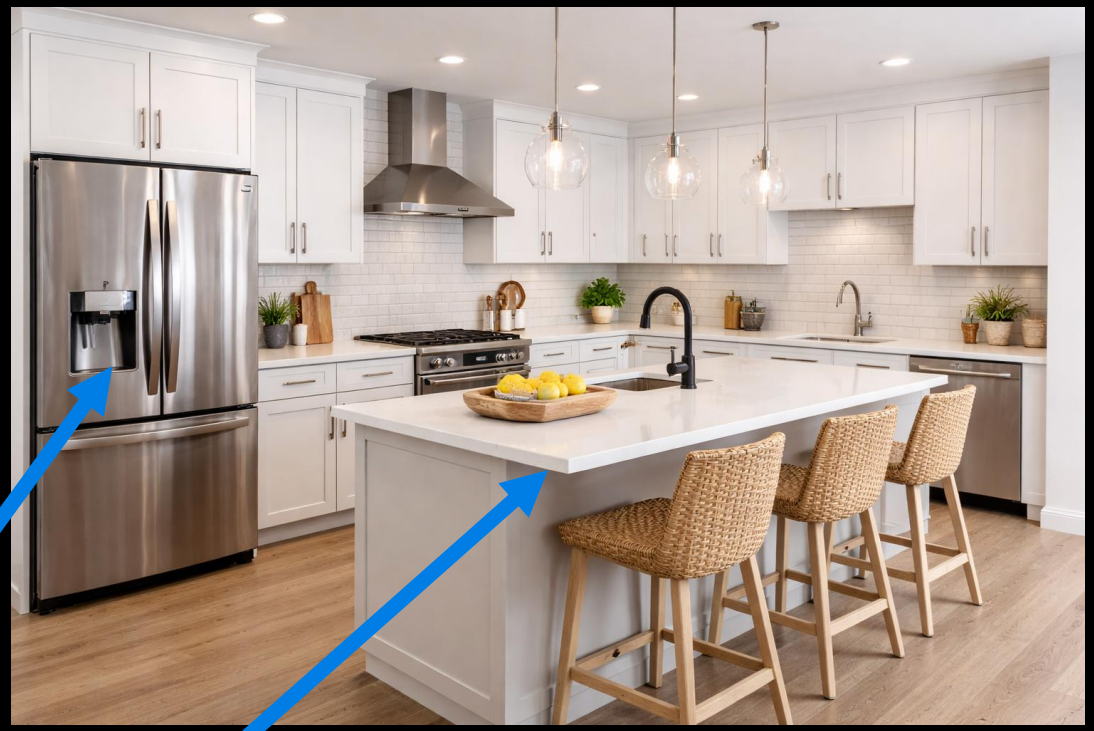
Working with files in Python

Why do we care about files?

- Everything we've done so far is essentially deleted when our program ends
- What if we wanted to store information?

Why work with files?

- While a program is running, its data is stored in random access memory (RAM). *when the program ends data in RAM disappears*.
- To make data available the next time the program is started, it must be written to a non-volatile storage medium, such a hard drive, USB drive, cloud storage, or CD-R.



Disk

RAM

Why work with files?

- Data on non-volatile storage media are stored in named locations on the media called **files**.
- By **reading** and **writing** files, programs can save information between program runs.

It's like working with a notebook!

- A file must be **opened**.
- When you are done, it has to be **closed**.
- While the file is open, it can either be **read from or written to**.
- Like a bookmark, the file **keeps track of** where you are reading to or writing from.
- You can read the whole file in its **natural order** or you can **skip** around.

The file class

Instances (objects) of the **file** class.

name: bills.txt
mode: append

name: grades.txt
mode: read

name: itinerary.txt
mode: write

File

filename
mode (ex: read or write)

read
readline
write
close
...

Opening a File

- Python has a built-in function `open` that creates and returns a file object with a connection between the file information on the disk and the program.
- The general form for opening a file is:

```
open(filename, mode)
```

where **mode** is `'r'` (to open for reading), `'w'` (to open for writing), or `'a'` (to open for appending to what is already in the file).

and **filename** is an **external** storage location.

Writing to a File

- The following statement opens the file `test.txt` in write mode `'w'`.

```
myfile = open('test.txt', 'w')
```

If there is no file named "`test.txt`" on the disk, it will be created. *If there already is one, it will be replaced by the file we are writing*.

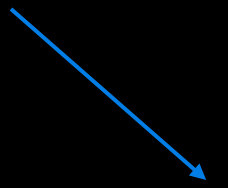
- The file information will be assigned to the file object `myfile`.

Writing to a File

- To write something we need to use the `write` method as shown:

```
myfile.write('CATS!...')
```

- The `write` method does not attach a new line character by default.



```
myfile.write('\n')
myfile.write('I <3 my second line... \n')
```

- Every time we use `myfile.write()` a string is added to file "`test.txt`" where we left off.

Closing a File

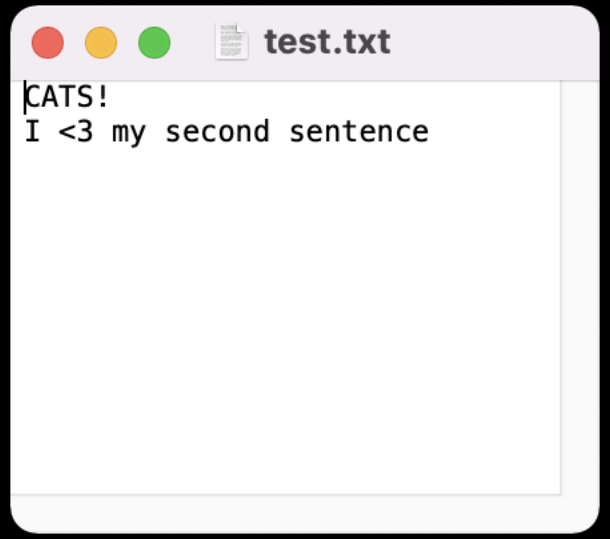
- Once you have finished with the file, you need to close it:

```
myfile.close()
```

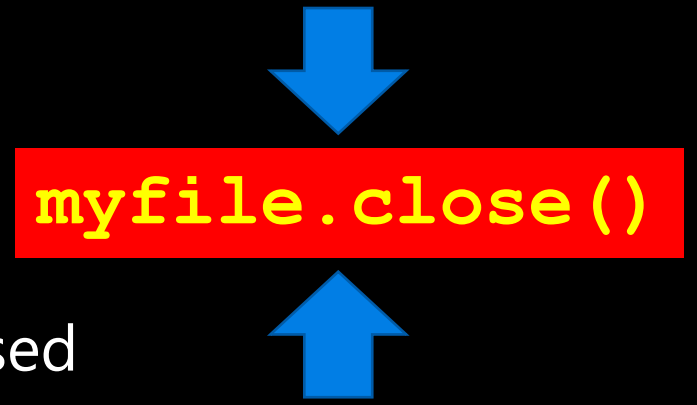
- This tells the system that we are done writing and makes the disk file available for reading or writing by other programs (or by our own program).

```
myfile = open('test.txt', 'w')
myfile.write('CATS!…')
myfile.write('\n')
myfile.write('I <3 my second sentence… \n')


# and eventually we close our file
myfile.close()
```



CLOSE THE FILE!



- Always close a file that you've opened!
  - Many changes don't occur until **after** the file is closed
  - Leaving open is a waste of a computer's resources
    - Slows down program, uses more space in RAM, impacts performance
  - Other files may treat the file as "open" and won't be able to read the file
  - Could run into limits about how many files you have open
  - Not #cleancode

The "Writing to Files" Recipe

go together

```
# open/create a file
myfile = open("grades.txt", "w")


# write to a file
myfile.write('string')


# close the file
myfile.close()
```

Let's Code!

- Let's take a look at how this works in Python!
  - Writing to your first file
  - Figure out where your files are getting saved!
    - Current working directory
  - Try playing with "w" and "a" mode!

In your notebook,

Click Link:
2. The File Object

- Madlibs is a story game
- A story is written, and a few important words are taken out, replaced by blanks
- The blanks are labelled with their part of speech or other category ("noun", "adjective", "an animal", and so on)
- Replace the categories with specific words without knowing the story
- When all the blanks have been filled in, the story is read out, usually with comic results
- Example: https://youtu.be/kM9Wuzj4k24

Let's Code!

- Let's take a look at how this works in Python!
  - Bringing it all together with Madlibs!

Open your notebook

Click Link:
4. Madlibs

APS106

UNIVERSITY OF TORONTO

introduction to Object Oriented Programming, and the file object.

Week 5 | Lecture 2 (5.2)

if nothing else, write #cleancode